



CAPÍTULO 6 DE 6

Modularização e Funções

Como dividir um programa em partes menores e reutilizáveis, usando funções para organizar melhor o código.



INSTITUTO FEDERAL
Sudeste de Minas Gerais

Educação a Distância
Pró-reitoria de Ensino

O que é modularização?

- 1 É dividir um programa grande em partes menores, chamadas módulos ou funções
- 2 Cada parte fica responsável por uma tarefa específica e bem definida
- 3 Facilita a organização, o entendimento e a manutenção do código

Vantagens de modularizar o código

- 1 Organização e clareza — cada função tem uma responsabilidade
- 2 Reutilização — evita repetir o mesmo código várias vezes
- 3 Facilidade de manutenção e correção de erros
- 4 Facilita o trabalho em equipe, dividindo tarefas
- 5 Permite testar cada parte do programa isoladamente

O que é uma função?

- 1 Bloco de código nomeado, que executa uma tarefa específica
- 2 Pode ser chamada (executada) quantas vezes forem necessárias
- 3 Em Python, uma função é criada com a palavra-chave def

```
def nome():
```

```
...
```

```
nome() # chamando a função
```

Sintaxe básica de uma função

def define a função; o código só executa quando ela é chamada pelo nome.

```
sintaxe_def.py

def nome_da_funcao():
    instrução
    instrução

nome_da_funcao()    # chamando a função
```

Exemplo: função simples

● ● ● saudacao.py

```
def saudacao():  
    print("Olá! Bem-vindo ao curso.")  
  
saudacao()  
saudacao()
```

SAÍDA

```
Olá! Bem-vindo ao curso.  
Olá! Bem-vindo ao curso.
```

Função com parâmetros

Parâmetros permitem que a função receba valores diferentes a cada chamada.

```
parameters.py

def saudacao(nome):
    print(f"Olá, {nome}!")

saudacao("Ana")
saudacao("Bruno")
```

SAÍDA

```
Olá, Ana!
Olá, Bruno!
```

Função com retorno: return

return devolve um valor para quem chamou a função.

retorno.py

```
def dobro(numero):  
    return numero * 2  
  
resultado = dobro(5)  
print(resultado)
```

SAÍDA

10

Exemplo: somar dois números

somar.py

```
def somar(a, b):  
    return a + b  
  
print(somar(3, 4))  
print(somar(10, 20))
```

SAÍDA

7
30

Exemplo: verificar par ou ímpar

par_impar.py

```
def par_ou_impar(numero):  
    if numero % 2 == 0:  
        return "Par"  
    else:  
        return "Ímpar"  
  
print(par_ou_impar(7))  
print(par_ou_impar(10))
```

SAÍDA

Ímpar
Par

Exemplo: área do círculo

```
area_circulo.py

def area_circulo(raio):
    pi = 3.14159
    return pi * raio ** 2

print(area_circulo(5))
```

SAÍDA

78.53975

Parâmetros com valor padrão

Se o segundo argumento não for informado, o valor padrão é usado.

```
valor_padrao.py

def saudacao(nome, mensagem="Bem-vindo") :
    print(f"{mensagem}, {nome}!")

saudacao("Ana")
saudacao("Bruno", "Boa tarde")
```

SAÍDA

```
Bem-vindo, Ana!
Boa tarde, Bruno!
```

Escopo de variáveis (local x global)

Variáveis criadas dentro de uma função só existem enquanto ela executa.

```
escopo.py

x = 10    # variável global

def teste():
    x = 5    # variável local
    print(x)

teste()
print(x)
```

SAÍDA

5
10

Combinando várias funções em um programa

Funções pequenas podem ser combinadas para formar um programa completo.

```
programa_completo.py

def entrada():
    return float(input("Digite um valor: "))

def dobro(n):
    return n * 2

def exibir(valor):
    print(f"O dobro é {valor}")

n = entrada()
resultado = dobro(n)
exibir(resultado)
```

Resumo do capítulo — e do curso

- 1 Modularização organiza o programa em partes menores e reutilizáveis
- 2 Funções: def, parâmetros e return são a base para criar suas próprias funções
- 3 Do algoritmo até as funções, cada capítulo constrói a base da lógica de programação

Capítulos do curso:

1. Lógica e algoritmos
2. Variáveis e dados
3. Condicionais
4. Repetição
5. Vetores e matrizes
6. Funções



CAPÍTULO 6 DE 6

Modularização e Funções

Como dividir um programa em partes menores e reutilizáveis, usando funções para organizar melhor o código.



INSTITUTO FEDERAL
Sudeste de Minas Gerais

Educação a Distância
Pró-reitoria de Ensino